LEVELUP CLOUD TRAININGS

levelupedu.net

PySpark RDD Masterclass

Module 5: Shared Variables, Shuffle & Actions

Student Class Notes

Trainer: Sateesh Pabbathi

Microsoft MVP | MCT | 22+ Years Experience | 5000+ Students Trained

Topics Covered:

Broadcast Variables • Accumulators • Shuffle • coalesce
saveAsTextFile • count() • countByKey() • Join

1. Shared Variables in PySpark

In PySpark, when you write a transformation using map() or filter(), PySpark ships a copy of every variable used inside that function to every task running on every executor. Think of it like giving every delivery boy at Zomato a separate printed copy of the restaurant menu — highly wasteful if the menu is 500 pages!

> **Sateesh Says**
> *Shared Variables solve a real-world distributed computing problem: How do you efficiently share data across all nodes without sending it millions of times? PySpark gives you two weapons: Broadcast Variables (read-only) and Accumulators (write-only aggregators).*

Why Do We Need Shared Variables?

Problem	Solution
Large lookup table sent to every task (wasteful)	Broadcast Variable — sent once per node
Counting errors/events across executors	Accumulator — safely aggregated back to driver
Driver variable modified inside task	Not visible back on driver — use Accumulator

2. Broadcast Variables

What are Broadcast Variables?

A Broadcast Variable allows PySpark to efficiently distribute a large read-only dataset to all nodes in the cluster. Instead of shipping a copy to every individual task, PySpark ships it ONCE to each executor node, and all tasks on that node share the same copy.

> **Real-World Analogy — HDFC Bank PIN Code Lookup**
> Imagine HDFC Bank has 10,000 PIN codes to city mapping. Instead of giving every bank teller (task) a separate copy of the 500-page list, you paste ONE copy on the wall of each branch office (executor node). All tellers in that branch read from that single shared list. That's exactly how a Broadcast Variable works.

Syntax & Usage

```
from pyspark import SparkContext
sc = SparkContext('local', 'BroadcastDemo')

# Large lookup table — e.g., city to zone mapping for Swiggy
```

```
city_zone_map = {
    'Hyderabad': 'South',
    'Mumbai': 'West',
    'Delhi': 'North',
    'Chennai': 'South'
}

# Broadcast it — sent ONCE per node, not per task
broadcast_map = sc.broadcast(city_zone_map)

# Create RDD of orders
orders = sc.parallelize([('HYD001', 'Hyderabad'), ('MUM002', 'Mumbai'), ('DEL003',
'Delhi')])

# Use the broadcast variable inside transformation
def get_zone(order):
    order_id, city = order
    zone = broadcast_map.value.get(city, 'Unknown')  # .value to access data
    return (order_id, city, zone)

result = orders.map(get_zone).collect()
print(result)
# Output: [('HYD001', 'Hyderabad', 'South'), ('MUM002', 'Mumbai', 'West'),
('DEL003', 'Delhi', 'North')]

# Cleanup
broadcast_map.unpersist()  # Free memory on executors
```

Key Rules

- Use .value to access the broadcasted data inside tasks
- Broadcast variables are READ-ONLY — you cannot modify them inside tasks
- Best for data that is large but fits in executor memory (few MBs to a few GBs)
- Call .unpersist() when done to free executor memory
- Call .destroy() to permanently remove from all executors and driver

> **Sateesh Says**
>
> *In interviews, they always ask: 'What is the difference between a normal variable and a Broadcast Variable?' Answer: Normal variable sends one copy per TASK. Broadcast variable sends one copy per NODE/Executor. If you have 100 tasks per node and 50 nodes = 5000 copies vs 50 copies. That's 100x less network traffic!*

When to Use Broadcast Variables

Use Case	Example
Large lookup table join	PIN code → city mapping for HDFC Bank
Configuration data	Tax rates, discount rules for Flipkart
ML model parameters	Feature coefficients for scoring pipeline
Reference data	Stock ticker → company name for NSE data

3. Accumulators

What are Accumulators?

Accumulators are variables that can ONLY be added to (not read inside tasks). They are used to aggregate values across all executors back to the driver. PySpark guarantees that the accumulator is updated exactly once per task (in case of retries, it handles deduplication).

> **Real-World Analogy — Zomato Order Error Counter**
>
> Imagine Zomato has 1000 delivery partners processing orders in parallel. Each partner (task) finds bad orders and wants to report the count back to HQ (driver). Instead of sending a message each time, they drop a counter token in a central box (accumulator). At the end of the shift, HQ counts all tokens. That's an Accumulator!

Syntax & Usage

```
from pyspark import SparkContext
sc = SparkContext('local', 'AccumulatorDemo')

# Create accumulators
error_count = sc.accumulator(0)      # Integer accumulator, starts at 0
total_amount = sc.accumulator(0.0)   # Float accumulator

# Sample order data: (order_id, amount, status)
orders = sc.parallelize([
    ('O001', 250.0, 'delivered'),
    ('O002', 0.0, 'error'),
    ('O003', 180.5, 'delivered'),
    ('O004', -50.0, 'error'),
    ('O005', 320.0, 'delivered')
])

def process_order(order):
    order_id, amount, status = order
    if status == 'error' or amount <= 0:
        error_count.add(1)        # Increment error counter
    else:
        total_amount.add(amount)  # Add to total
    return order

# Trigger action (accumulators update only during actions)
result = orders.map(process_order).collect()

# Read accumulator value ONLY on driver after action
print(f'Total Errors: {error_count.value}')     # Total Errors: 2
print(f'Total Revenue: {total_amount.value}')   # Total Revenue: 750.5
```

Critical Rules for Accumulators

- Accumulators can only be READ on the driver — reading inside tasks gives unreliable results
- Accumulators are WRITE-ONLY inside tasks — use .add() or += to update
- PySpark only guarantees accumulator updates inside actions (collect, count, etc.)
- In transformations (map, filter), updates may be applied multiple times on retries
- Use for counting errors, tracking records processed, summing metrics

Sateesh Says

Common interview trap: 'Can you read an accumulator inside a map() transformation?' Technically yes, but DON'T. PySpark may re-execute tasks on failure and the value will be wrong. Always read accumulators AFTER an action is triggered — that's the only safe time.

Broadcast vs Accumulator — Quick Comparison

Property	Broadcast Variable vs Accumulator
Direction	Broadcast: Driver → Executors | Accumulator: Executors → Driver
Access in tasks	Broadcast: Read-only | Accumulator: Write-only (add only)
Purpose	Broadcast: Share lookup data | Accumulator: Aggregate metrics
Result read	Broadcast: .value (anywhere) | Accumulator: .value (driver only)
Example	Broadcast: City-State map | Accumulator: Error counter

4. Shuffle in PySpark

What is Shuffle?

Shuffle is the process of redistributing data across partitions — often across executor nodes — so that records with the same key end up in the same partition. It is one of the most expensive operations in PySpark because it involves disk I/O, network transfer, and serialization.

> **Real-World Analogy — Bank Branch Consolidation**
> Imagine Axis Bank wants to calculate total account balance per city. Customers (data) are spread across 100 branches (partitions). To get city totals, all records from Hyderabad branches must be brought to ONE branch for counting. This physical movement of records across branches = Shuffle.

What Triggers Shuffle?

- groupByKey() — groups all values for each key
- reduceByKey() — aggregates values per key
- join() — combines two RDDs on matching keys
- sortByKey() — sorts records by key globally
- distinct() — removes duplicates across partitions
- repartition() — redistributes data into N partitions

Shuffle Lifecycle

```
# Example: Total sales per city for Flipkart
sales = sc.parallelize([
    ('Hyderabad', 5000), ('Mumbai', 8000), ('Hyderabad', 3000),
    ('Delhi', 6000), ('Mumbai', 2000), ('Delhi', 4000)
])

# reduceByKey triggers SHUFFLE — data moves across nodes
city_sales = sales.reduceByKey(lambda a, b: a + b)
print(city_sales.collect())
# Output: [('Hyderabad', 8000), ('Mumbai', 10000), ('Delhi', 10000)]
```

reduceByKey vs groupByKey — Shuffle Impact

Aspect	reduceByKey vs groupByKey
Pre-shuffle aggregation	reduceByKey: Yes (combines locally first) | groupByKey: No
Network data transfer	reduceByKey: Less (pre-aggregated) | groupByKey: More (all raw values)
Memory usage	reduceByKey: Lower | groupByKey: Higher (all values in memory)
Performance	reduceByKey: Faster | groupByKey: Slower
Use when	reduceByKey: Sum/Count | groupByKey: Need all values (rare)

> **Sateesh Says**
> *In every Spark interview, they ask: 'Why is groupByKey bad?' Because it sends ALL values across the network first, THEN aggregates. reduceByKey aggregates locally (map-side) first, then sends only the partial result. For 1 billion Swiggy orders, the difference is MASSIVE. Always prefer reduceByKey!*

5. coalesce Transformation

What is coalesce?

coalesce(n) reduces the number of partitions in an RDD to n WITHOUT triggering a full shuffle. It is an optimized version of repartition() when you want to DECREASE the number of partitions.

> **Real-World Analogy — Merging Bank Files**
> Imagine ICICI Bank processes data in 100 files daily but wants to write the final output as 5 files. coalesce merges partitions locally (nearby workers combine their data) rather than reshuffling everything across the network. It's like merging 5 departments in the same office floor vs moving people from different cities.

```
# Initial RDD with many partitions
data = sc.parallelize(range(1, 101), 20)  # 20 partitions
print(f'Initial partitions: {data.getNumPartitions()}')  # 20

# coalesce — reduces partitions without full shuffle
reduced = data.coalesce(5)
print(f'After coalesce: {reduced.getNumPartitions()}')  # 5

# repartition — triggers full shuffle (use when INCREASING partitions)
increased = data.repartition(50)
print(f'After repartition: {increased.getNumPartitions()}')  # 50

# Practical use: before saving to reduce output files
result_rdd = sales.reduceByKey(lambda a, b: a + b).coalesce(1)
result_rdd.saveAsTextFile('/output/city_sales')
```

coalesce vs repartition

Property	coalesce vs repartition
Shuffle triggered	coalesce: No (or minimal) | repartition: Yes (full shuffle)
Direction	coalesce: Decrease only | repartition: Increase or Decrease
Speed	coalesce: Faster | repartition: Slower
Data skew risk	coalesce: Yes (uneven) | repartition: No (even distribution)

When to use	coalesce: Before write (reduce files) | repartition: Before joins/aggregations

6. saveAsTextFile

What is saveAsTextFile?

saveAsTextFile is an action that saves the RDD contents to a text file (or set of files) on a distributed file system like HDFS or local disk. Each partition creates one output file.

```
# Basic saveAsTextFile
sales_rdd = sc.parallelize([
    'Hyderabad,Electronics,50000',
    'Mumbai,Clothing,30000',
    'Delhi,Furniture,45000'
])

# Save to local path — each partition = one part file
sales_rdd.saveAsTextFile('/tmp/sales_output')
# Creates: /tmp/sales_output/part-00000, part-00001, ...

# Save with compression (production best practice)
from pyspark.io import GzipCodec
sales_rdd.saveAsTextFile('/tmp/sales_gz', GzipCodec)

# Best practice: coalesce before saving to control file count
sales_rdd.coalesce(1).saveAsTextFile('/tmp/single_output')
# Creates only: /tmp/single_output/part-00000
```

> **Sateesh Says**
>
> *In production at companies like Infosys or TCS, you never coalesce(1) on large datasets because it creates a bottleneck. Use coalesce(5) or coalesce(10) depending on data size. Rule of thumb: each output file should be 128MB–256MB for HDFS efficiency.*

7. count() Action

What is count()?

count() is an action that returns the total number of elements in the RDD. It triggers a full traversal of all partitions and returns a single integer to the driver.

```
# Basic count
orders = sc.parallelize([
    ('O001', 'Hyderabad', 250.0),
    ('O002', 'Mumbai', 0.0),
    ('O003', 'Delhi', 180.5),
    ('O004', 'Hyderabad', 320.0)
])

total_orders = orders.count()
print(f'Total Orders: {total_orders}')  # Total Orders: 4
```

```
# Count after filter
valid_orders = orders.filter(lambda x: x[2] > 0).count()
print(f'Valid Orders: {valid_orders}')  # Valid Orders: 3

# countApprox — faster for huge datasets (returns estimate)
approx = orders.countApprox(timeout=1000, confidence=0.95)
print(f'Approx Count: {approx}')  # ~4
```

Sateesh Says

count() always triggers a job and scans all partitions. If your RDD is already cached (persist/cache), count() is fast. If not cached, every count() call re-executes the full lineage. Cache your RDD before calling count() multiple times!

8. countByKey() Action

What is countByKey()?

countByKey() is an action that counts the number of elements for each key in a key-value (pair) RDD. It returns a Python dictionary to the driver with key → count mapping.

Real-World Analogy — Swiggy Orders per City

Think of countByKey() as Swiggy's end-of-day report: How many orders came from each city? Each order is a (city, order_details) pair. countByKey() gives you {'Hyderabad': 1523, 'Mumbai': 2108, 'Delhi': 987} — exactly what the operations team needs.

```
# Sample: Orders per city for Swiggy
orders = sc.parallelize([
    ('Hyderabad', 'Biryani'),
    ('Mumbai', 'Pizza'),
    ('Hyderabad', 'Dosa'),
    ('Delhi', 'Burger'),
    ('Mumbai', 'Pasta'),
    ('Hyderabad', 'Noodles'),
    ('Delhi', 'Sandwich')
])

# countByKey returns a defaultdict (Python dict)
city_counts = orders.countByKey()
print(dict(city_counts))
# Output: {'Hyderabad': 3, 'Mumbai': 2, 'Delhi': 2}

# Practical: Find busiest city
busiest = max(city_counts, key=city_counts.get)
print(f'Busiest City: {busiest} with {city_counts[busiest]} orders')
# Output: Busiest City: Hyderabad with 3 orders

# Alternative using reduceByKey (scalable for huge datasets)
# countByKey collects everything to driver — careful with large cardinality!
city_counts_rdd = orders.map(lambda x: (x[0], 1)).reduceByKey(lambda a, b: a + b)
```

```
print(city_counts_rdd.collect())
```

countByKey() — Important Warning

Aspect	Details
Return type	Python defaultdict — all data collected to DRIVER
Use when	Number of distinct keys is small (fits in driver memory)
Avoid when	Millions of unique keys — use reduceByKey().collect() instead
Equivalent to	map(lambda x: (x[0], 1)).reduceByKey(add).collectAsMap()

Sateesh Says

countByKey() is a convenience method — it works beautifully for small cardinality (few hundred keys like cities, categories). But if you're counting 50 million unique user IDs at Flipkart, countByKey() will crash your driver. Use reduceByKey() instead for large-scale key counting!

9. Join in PySpark RDD

What is Join?

join() in PySpark RDD combines two pair RDDs by matching on keys, similar to SQL INNER JOIN. It returns a new RDD of (key, (value1, value2)) tuples for all matching keys.

```
# Scenario: Join Customer data with Order data (like Flipkart)

# Customers: (customer_id, customer_name)
customers = sc.parallelize([
    (1, 'Rahul'),
    (2, 'Priya'),
    (3, 'Arjun')
])

# Orders: (customer_id, order_amount)
orders = sc.parallelize([
    (1, 1500.0),
    (2, 3200.0),
    (1, 850.0),
    (4, 500.0)   # customer_id 4 not in customers
])

# INNER JOIN — only matching keys
joined = customers.join(orders)
print(joined.collect())
# Output: [(1, ('Rahul', 1500.0)), (1, ('Rahul', 850.0)), (2, ('Priya', 3200.0))]
# Note: customer_id 3 (Arjun) and 4 are excluded

# LEFT OUTER JOIN — all customers, even with no orders
left = customers.leftOuterJoin(orders)
print(left.collect())
# Output: [(1, ('Rahul', 1500.0)), (1, ('Rahul', 850.0)),
#          (2, ('Priya', 3200.0)), (3, ('Arjun', None))]

# FULL OUTER JOIN — all records from both
full = customers.fullOuterJoin(orders)
```

Join Types Comparison

Join Type	Description
join()	INNER JOIN — only keys present in BOTH RDDs
leftOuterJoin()	LEFT JOIN — all keys from left RDD, None for missing right
rightOuterJoin()	RIGHT JOIN — all keys from right RDD, None for missing left
fullOuterJoin()	FULL OUTER — all keys from both, None where missing
subtractByKey()	Returns keys in left RDD not present in right RDD

Sateesh Says

Join triggers a SHUFFLE — PySpark must move all records with the same key to the same partition. If one RDD is very small (like a lookup table with 500 cities), use Broadcast Join instead: broadcast the small RDD and do a map-side join. No shuffle needed = 10x faster!

10. Interview Questions & Answers

Question	Answer
What is a Broadcast Variable?	A read-only variable sent once per executor node (not per task) to efficiently share large data across the cluster.
When should you use Broadcast Variables?	When you have a large lookup table (e.g., PIN-to-city map) that needs to be accessed in every task but doesn't change.
Can you modify a Broadcast Variable in a task?	No. Broadcast variables are strictly read-only. Modifications inside tasks are not reflected anywhere.
What is an Accumulator?	A variable that tasks can only add to (not read), which is then aggregated back to the driver. Used for counting events or summing metrics.
When is it safe to read an Accumulator?	Only on the driver, AFTER an action (like collect() or count()) has been triggered.
What is Shuffle? Why is it expensive?	Shuffle is the redistribution of data across partitions/nodes. It's expensive because it involves disk I/O, network transfer, and serialization.
reduceByKey vs groupByKey — which is better?	reduceByKey is better — it does local aggregation first (map-side combine) before shuffling, sending less data over the network.
What is coalesce? When to use it?	coalesce(n) reduces partition count without full shuffle. Use before saveAsTextFile to reduce output file count.
What does countByKey return?	A Python defaultdict with key-count pairs. Caution: collects all data to driver — avoid for high-cardinality keys.
What join types are available in RDD?	join() (inner), leftOuterJoin(), rightOuterJoin(), fullOuterJoin(), subtractByKey()